QUALITY REPORT

Sample Shop

Generated 1 Nov 2026, 18:51 UTC · Prepared for centinel-local-qa-da362620@example.com

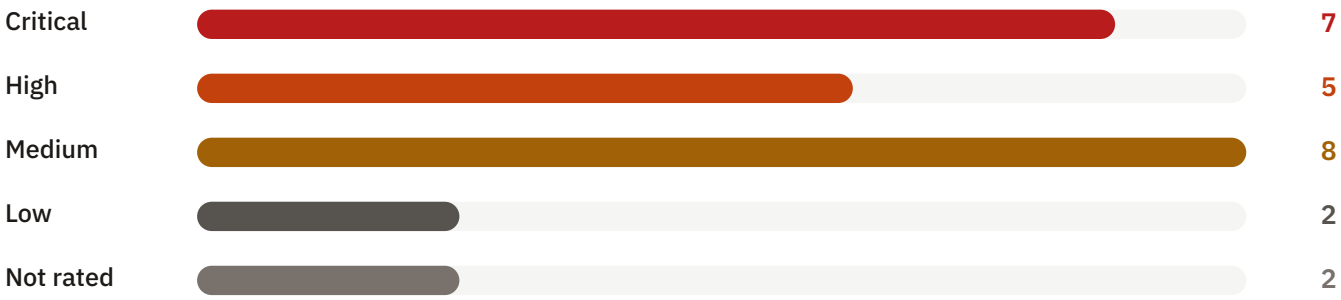
Tiny web shop from examples/sample-shop, used to try Centinel.

Fix before release

Centinel found 12 serious issues (critical or high risk) that are still open.

Open issues	Serious issues	Code review	Browser test
24	12	Needs approval	Passed
still to look at	critical or high risk	1 Nov 2026	1 Nov 2026

Open issues by risk level



Critical and high issues should be fixed before you release. Medium and low issues can be planned.

What Centinel checked

Code review	"Pre-release review (Retry)". Checked your project sources. It is waiting for someone to approve it.	Needs approval
Browser test	"Sign-in page shows the login form". The website did what the test expected. Recorded 1 step. Website: http://localhost:4100	Passed
Requirements	0 requirements implemented · 0 partly implemented · 16 not linked to code yet	

What to do next

1. Fix "Broken Object-level Authorization for Orders": Scope the order lookup to both the requested order ID and the authenticated user's ID. For absent or non-owned orders, return exactly "Order not foun...
2. Fix "Credential Enumeration": Return exactly "Email or password is incorrect." for both unknown-email and wrong-password failures.
3. Open the latest code review and approve it or request changes.

Top issues explained

The most important open issues, highest risk first.

Critical risk Code review

Broken Object-level Authorization for Orders

WHAT IS WRONG

The documented order endpoint returns any order to any signed-in user, including another user's shipping address. Authentication without an ownership check exposes private order details and violates the order-privacy requirements.

HOW TO FIX IT

Scope the order lookup to both the requested order ID and the authenticated user's ID. For absent or non-owned orders, return exactly "Order not found." without any order fields or ownership-distinguishing response.

WHERE

`README.md:30`

Critical risk Code review

Credential Enumeration

WHAT IS WRONG

The login endpoint returns a distinct message when an email is absent, while a wrong password returns the required generic message. This allows callers to determine which email addresses are registered.

HOW TO FIX IT

Return exactly "Email or password is incorrect." for both unknown-email and wrong-password failures.

WHERE

`server.js:45`

Critical risk Code review

Missing Order Ownership Check

WHAT IS WRONG

The order-view route authenticates the caller but returns any order found by ID without comparing order.userId with the authenticated userId. A signed-in user who supplies another user's order ID receives that order's details, including the seeded shipping address.

HOW TO FIX IT

Return exactly "Order not found." unless an order exists and order.userId equals the authenticated userId; do not serialize any order fields before that check.

WHERE

`server.js:76`

Critical risk Code review

Order Ownership Check Missing

WHAT IS WRONG

The authenticated order-view route looks up an order by ID and returns it without verifying that order.userId matches the signed-in userId. Any signed-in user who knows another order ID can receive that order's details, including the seeded shipping address, contrary to the order-privacy requirements.

HOW TO FIX IT

Before returning an order, require both that it exists and that order.userId equals the authenticated userId. Return exactly `{ error: 'Order not found.' }` with 404 for either a missing or foreign order, and add cross-user access tests verifying that no order fields are disclosed.

WHERE

`server.js:76`

Critical risk Code review

[secrets-api-key] Possible hardcoded credential in "password". Verify whether this literal is sensitive and reachable before treating it as a leak.

WHAT IS WRONG

Possible hardcoded credential in "password". Verify whether this literal is sensitive and reachable before treating it as a leak.

HOW TO FIX IT

Review and address this secrets finding.

WHERE

`src/users.js:3`

+ 13 more open issues, listed in Part 2: Technical details.

Browser test result

Sign-in page shows the login form

Passed

What it checked	Open the shop and confirm that the sign-in form and the product list are shown. Do not sign in.
Website	http://localhost:4100
Result	The website did what the test expected. Recorded 1 step.
Summary	Revision 1: 1 of 1 checks passed; verdict passed.

What the risk levels mean

Critical	Could cause a security breach, data loss or a broken core feature. Fix before release.
High	Likely to break an important feature or expose data. Fix before release.
Medium	Causes problems in some situations. Plan a fix soon.
Low	Minor issue or cleanup. Fix when convenient.

PART 2

Technical details

For developers and auditors: exact IDs, versions and timestamps, the full finding history, evidence and model usage.

About this report

Project	Sample Shop
Project ID	6c1827b1-9903-4f45-97f8-99e429e879dc
Report ID	b1af2f88-6afc-42b8-8c7d-1a47c8b1dfb7
Generated	2026-11-01T18:51:00.442Z
Generated by	centinel-local-qa-da362620@example.com
Report format version	project-report-v4
Risk policy version	centinel-risk-v1

Risk assessment

Current static risk items	19
Critical / high	5 / 5
Medium / low	8 / 1
Unclassified	0

Dynamic findings are included in the inventory but remain outside this static risk policy.

Current risk items

- critical - Order Ownership Check Missing (New)
- critical - Missing Order Ownership Check (New)
- critical - [secrets-api-key] Possible hardcoded credential in "password". Verify whether this literal is sensitive and reachable before treating it as a leak. (New)
- critical - Broken Object-level Authorization for Orders (New)
- critical - Credential Enumeration (New)

Current project findings

1. Order Ownership Check Missing

Canonical finding ID	bf61ded6-9798-42b1-8d8d-4123a28fb176
Source module	static
Persisted status	New
Effective disposition	New
Risk level	critical
Location	server.js:76
Latest occurrence	Canonical finding
Lineage references	None

2. Missing Order Ownership Check

Canonical finding ID	af9c66cd-b5b8-435a-9165-dc228966b043
Source module	static

Persisted status	New
Effective disposition	New
Risk level	critical
Location	server.js:76
Latest occurrence	Canonical finding
Lineage references	None

3. Server-Side Quantity Validation Missing

Canonical finding ID	9682a57f-90f2-419f-b899-4754aa559a00
Source module	static
Persisted status	New
Effective disposition	New
Risk level	high
Location	server.js:65
Latest occurrence	Canonical finding
Lineage references	None

4. Missing Order Ownership Check

Canonical finding ID	47033209-ccaf-412f-8075-ef9ed93b329c
Source module	static
Persisted status	New
Effective disposition	New
Risk level	critical
Location	server.js:76
Latest occurrence	Canonical finding
Lineage references	None

5. Missing Server-Side Quantity Validation

Canonical finding ID	821bad2d-85a5-407b-ab73-f6603cf454dc
Source module	static
Persisted status	New
Effective disposition	New
Risk level	high
Location	server.js:65
Latest occurrence	Canonical finding
Lineage references	None

6. [secrets-api-key] Possible hardcoded credential in "password". Verify whether this literal is sensitive and reachable before treating it as a leak.

Canonical finding ID	ef77f5a6-90ea-4af0-b090-86c5e2ea1707
Source module	static
Persisted status	New
Effective disposition	New
Risk level	critical
Location	src/users.js:3
Latest occurrence	Canonical finding
Lineage references	None

7. Broken Object-level Authorization for Orders

Canonical finding ID	5945ff73-7b79-40ee-8fe9-d6d4c20d01a5
Source module	static
Persisted status	New
Effective disposition	New
Risk level	critical
Location	README.md:30
Latest occurrence	Canonical finding
Lineage references	None

8. Missing Server-side Quantity Validation

Canonical finding ID	158c5754-c8b8-4b38-afe0-fae35c1d226a
Source module	static
Persisted status	New
Effective disposition	New
Risk level	high
Location	README.md:29
Latest occurrence	Canonical finding
Lineage references	None

9. Missing Server-Side Quantity Validation

Canonical finding ID	307441bb-3a88-4e15-a472-52b1ac53f46c
Source module	static
Persisted status	New
Effective disposition	New
Risk level	high
Location	server.js:65
Latest occurrence	Canonical finding
Lineage references	None

10. Missing Server-Side Quantity Validation

Canonical finding ID	68b96592-de7b-460a-9545-f106f2918561
Source module	static
Persisted status	New
Effective disposition	New
Risk level	high
Location	server.js:65
Latest occurrence	Canonical finding
Lineage references	None

11. Credential Enumeration

Canonical finding ID	41fcd867-309d-447b-bb88-b73f6f9f0561
Source module	static
Persisted status	New
Effective disposition	New
Risk level	critical

Location	server.js:45
Latest occurrence	Canonical finding
Lineage references	None

12. [secrets-api-key] Possible hardcoded credential in "password". Verify whether this literal is sensitive and reachable before treating it as a leak.

Canonical finding ID	95027dcd-11ea-4907-9a5f-0ce6afeb5bb7
Source module	static
Persisted status	New
Effective disposition	New
Risk level	critical
Location	src/users.js:3
Latest occurrence	Canonical finding
Lineage references	None

13. Published Demo Credentials

Canonical finding ID	decb3276-4e95-4284-88b8-c2402ba33a74
Source module	static
Persisted status	New
Effective disposition	New
Risk level	medium
Location	README.md:17
Latest occurrence	Canonical finding
Lineage references	None

14. Unsupported Discount Variants Accepted

Canonical finding ID	6b096816-f891-481f-a1f2-c60ffbd893df
Source module	static
Persisted status	New
Effective disposition	New
Risk level	medium
Location	src/orders.js:25
Latest occurrence	Canonical finding
Lineage references	None

15. Credential Error Enables Email Enumeration

Canonical finding ID	4b295795-8178-4ff4-88a7-2bca9d2f93f4
Source module	static
Persisted status	New
Effective disposition	New
Risk level	medium
Location	server.js:45
Latest occurrence	Canonical finding
Lineage references	None

16. Unvalidated Email Input Can Throw

Canonical finding ID	abd301b2-5da4-481f-a4b0-c5f6b6bacf0c

Source module	static
Persisted status	New
Effective disposition	New
Risk level	medium
Location	src/users.js:7
Latest occurrence	Canonical finding
Lineage references	None

17. User Enumeration Through Distinct Login Error

Canonical finding ID	3921f473-7df8-42b1-9853-087c9a9a73d6
Source module	static
Persisted status	New
Effective disposition	New
Risk level	medium
Location	server.js:45
Latest occurrence	Canonical finding
Lineage references	None

18. Account Enumeration via Distinct Sign-in Error

Canonical finding ID	66832dd5-2e2d-40d7-b119-2fe33b16404c
Source module	static
Persisted status	New
Effective disposition	New
Risk level	medium
Location	README.md:28
Latest occurrence	Canonical finding
Lineage references	None

19. Unsupported Discount Variants Accepted

Canonical finding ID	d59709c1-1e80-4470-b8b0-6369ffdbaf9e
Source module	static
Persisted status	New
Effective disposition	New
Risk level	medium
Location	src/orders.js:24
Latest occurrence	Canonical finding
Lineage references	None

20. Account Enumeration Through Distinct Login Error

Canonical finding ID	a11306f4-ef8b-4471-9ef9-ff875f89faac
Source module	static
Persisted status	New
Effective disposition	New
Risk level	medium
Location	server.js:45
Latest occurrence	Canonical finding
Lineage references	None

21. [cq-console-log] Possible debug output call. Check its purpose against the project logging policy.

Canonical finding ID	50093a63-837f-480a-9c19-9e9d6d42990e
Source module	static
Persisted status	New
Effective disposition	New
Risk level	low
Location	server.js:107
Latest occurrence	Canonical finding
Lineage references	None

22. [cq-console-log] Possible debug output call. Check its purpose against the project logging policy.

Canonical finding ID	ff2e0d0c-76ee-40bf-a5b2-801927241620
Source module	static
Persisted status	New
Effective disposition	New
Risk level	low
Location	server.js:107
Latest occurrence	Canonical finding
Lineage references	None

23. [sec-innerhtml] HTML injection sink needs flow verification. This assignment alone does not establish an exploitable XSS vulnerability.

Canonical finding ID	89e1ff6a-c4d7-4cc7-8368-8cc270885a46
Source module	static
Persisted status	New
Effective disposition	New
Risk level	Not classified
Location	public/app.js:19
Latest occurrence	Canonical finding
Lineage references	None

24. [sec-innerhtml] HTML injection sink needs flow verification. This assignment alone does not establish an exploitable XSS vulnerability.

Canonical finding ID	6e0d545f-2eb3-4295-85fa-87edabf8faf1
Source module	static
Persisted status	New
Effective disposition	New
Risk level	Not classified
Location	public/app.js:19
Latest occurrence	Canonical finding
Lineage references	None

Historical finding occurrences

Saved states in this section are immutable historical occurrences and may repeat one canonical issue. Per-Review findings remain session-scoped.

1. Order Ownership Check Missing

Risk	critical
Severity / priority	critical / high
State	New
Module / category	static / authorization
Finding ID	bf61ded6-9798-42b1-8d8d-4123a28fb176
Location	server.js:76
Created	2026-11-01T18:32:36.106Z

The authenticated order-view route looks up an order by ID and returns it without verifying that order.userId matches the signed-in userId. Any signed-in user who knows another order ID can receive that order's details, including the seeded shipping address, contrary to the order-privacy requirements.

Evidence

```
const order = findOrder(orderMatch[1]);
  if (!order) return sendJson(res, 404, { error: 'Order not found.' });
  return sendJson(res, 200, { order });
```

Recommendation: Before returning an order, require both that it exists and that order.userId equals the authenticated userId. Return exactly `{ error: 'Order not found.' }` with 404 for either a missing or foreign order, and add cross-user access tests verifying that no order fields are disclosed.

2. Missing Order Ownership Check

Risk	critical
Severity / priority	critical / high
State	New
Module / category	static / authorization
Finding ID	af9c66cd-b5b8-435a-9165-dc228966b043
Location	server.js:76
Created	2026-11-01T18:23:57.096Z

The order-view route authenticates the caller but returns any order found by ID without comparing order.userId with the authenticated userId. A signed-in user who supplies another user's order ID receives that order's details, including the seeded shipping address.

Evidence

```
const order = findOrder(orderMatch[1]);
```

Recommendation: Return exactly "Order not found." unless an order exists and order.userId equals the authenticated userId; do not serialize any order fields before that check.

3. Server-Side Quantity Validation Missing

Risk	high
Severity / priority	high / medium
State	New
Module / category	static / input_validation
Finding ID	9682a57f-90f2-419f-b899-4754aa559a00
Location	server.js:65
Created	2026-11-01T18:32:36.170Z

Checkout converts each client-supplied quantity with Number() and stores it without checking that it is an integer from 1 through 10. Negative, fractional, excessive, or non-finite quantities can therefore reach order total calculation and corrupt the resulting order total.

Evidence

```
lines.push({ productId: product.id, name: product.name, price: product.price, quantity: Number(item.quantity) });
```

Recommendation: Parse each quantity once and reject it unless `Number.isInteger(quantity)` and `quantity >= 1 && quantity <= 10`. Perform this validation before creating any order and add server tests for zero, negative, fractional, over-10, NaN, and infinite values.

4. Missing Order Ownership Check

Risk	critical
Severity / priority	high / high
State	New
Module / category	static / authorization
Finding ID	47033209-ccaf-412f-8075-ef9ed93b329c
Location	server.js:76
Created	2026-11-01T18:32:35.868Z

The order-view route verifies only that some user is signed in. It retrieves an order solely by request-controlled ID and returns it without comparing order.userId to the authenticated user, exposing another user's line items, total, and potentially shipping details instead of returning exactly "Order not found.".

Evidence

```
const order = findOrder(orderMatch[1]);
  if (!order) return sendJson(res, 404, { error: 'Order not found.' });
  return sendJson(res, 200, { order });
```

Recommendation: After lookup, treat the order as not found unless order.userId exactly matches the authenticated userId, and return only the required 404 error without order details.

5. Missing Server-Side Quantity Validation

Risk	high
Severity / priority	high / medium
State	New
Module / category	static / checkout validation
Finding ID	821bad2d-85a5-407b-ab73-f6603cf454dc
Location	server.js:65
Created	2026-11-01T18:32:35.796Z

Checkout converts the submitted quantity to a Number but does not reject fractional, non-finite, zero, negative, or greater-than-ten values before creating the order. Crafted requests can therefore place orders with quantities outside the required whole-number range of 1 through 10 and can corrupt calculated totals.

Evidence

```
lines.push({ productId: product.id, name: product.name, price: product.price, quantity: Number(item.quantity) });
```

Recommendation: Before adding each line, require Number.isInteger(quantity) and quantity >= 1 && quantity <= 10; otherwise return a deterministic 400 response without creating an order.

6. [secrets-api-key] Possible hardcoded credential in "password". Verify whether this literal is sensitive and reachable before treating it as a leak.

Risk	critical
Severity / priority	high / high
State	New
Module / category	static / secrets
Finding ID	ef77f5a6-90ea-4af0-b090-86c5e2ea1707

Location	src/users.js:3
Created	2026-11-01T18:32:35.620Z

Possible hardcoded credential in "password". Verify whether this literal is sensitive and reachable before treating it as a leak.

Evidence

password = [literal redacted]

Recommendation: Review and address this secrets finding.

7. Broken Object-level Authorization for Orders

Risk	critical
Severity / priority	high / high
State	New
Module / category	static / authorization_privacy
Finding ID	5945ff73-7b79-40ee-8fe9-d6d4c20d01a5
Location	README.md:30
Created	2026-11-01T18:23:57.650Z

The documented order endpoint returns any order to any signed-in user, including another user's shipping address. Authentication without an ownership check exposes private order details and violates the order-privacy requirements.

Evidence

| REQ-4 Order privacy | `GET /api/orders/:id` returns any order, including another user's shipping address, to any signed-in user. |

Recommendation: Scope the order lookup to both the requested order ID and the authenticated user's ID. For absent or non-owned orders, return exactly "Order not found." without any order fields or ownership-distinguishing response.

8. Missing Server-side Quantity Validation

Risk	high
Severity / priority	high / medium
State	New
Module / category	static / checkout_validation
Finding ID	158c5754-c8b8-4b38-afe0-fae35c1d226a
Location	README.md:29
Created	2026-11-01T18:23:57.563Z

The documented checkout implementation does not enforce the required quantity range on the server. A crafted negative quantity can produce a negative order total, violating both quantity validation and total-integrity requirements.

Evidence

| REQ-3 Checkout | The server never checks that quantity is 1–10, so a request with a negative quantity creates a negative total. |

Recommendation: Before calculating or persisting an order, reject every item whose quantity is not an integer from 1 through 10. Calculate totals only from server-authoritative product prices and validated quantities.

9. Missing Server-Side Quantity Validation

Risk	high
Severity / priority	high / medium
State	New
Module / category	static / checkout_validation
Finding ID	307441bb-3a88-4e15-a472-52b1ac53f46c
Location	server.js:65

Created	2026-11-01T18:23:57.246Z

Checkout converts each client-supplied quantity with `Number` and passes it into order creation without checking that it is an integer from 1 through 10. Order creation then uses that unchecked value in the subtotal, so fractional, negative, excessive, or non-finite quantities can affect stored order totals despite the browser-side cap.

Evidence

```
lines.push({ productId: product.id, name: product.name, price: product.price, quantity: Number(item.quantity) });
```

Recommendation: Before creating any order, require each submitted quantity to satisfy `Number.isInteger(quantity) && quantity >= 1 && quantity <= 10`; reject the entire request otherwise. Keep product prices sourced from the server catalogue.

10. Missing Server-Side Quantity Validation

Risk	high
Severity / priority	high / medium
State	New
Module / category	static / input validation
Finding ID	68b96592-de7b-460a-9545-f106f2918561
Location	server.js:65
Created	2026-11-01T18:23:57.001Z

Checkout converts the submitted quantity to a number but does not require an integer from 1 through 10 before creating the order. Negative, fractional, zero, or non-finite quantities can therefore affect the stored total.

Evidence

```
lines.push({ productId: product.id, name: product.name, price: product.price, quantity: Number(item.quantity) });
```

Recommendation: Before creating any order, reject each quantity unless Number.isInteger(quantity) is true and quantity is between 1 and 10 inclusive.

11. Credential Enumeration

Risk	critical
Severity / priority	high / high
State	New
Module / category	static / authentication privacy
Finding ID	41fcd867-309d-447b-bb88-b73f6f9f0561
Location	server.js:45
Created	2026-11-01T18:23:56.855Z

The login endpoint returns a distinct message when an email is absent, while a wrong password returns the required generic message. This allows callers to determine which email addresses are registered.

Evidence

```
if (!user) return sendJson(res, 401, { error: 'No account uses this email.' });
```

Recommendation: Return exactly "Email or password is incorrect." for both unknown-email and wrong-password failures.

12. [secrets-api-key] Possible hardcoded credential in "password". Verify whether this literal is sensitive and reachable before treating it as a leak.

Risk	critical
Severity / priority	high / high
State	New
Module / category	static / secrets
Finding ID	95027dcd-11ea-4907-9a5f-0ce6afeb5bb7
Location	src/users.js:3

Created	2026-11-01T18:23:56.798Z

Possible hardcoded credential in "password". Verify whether this literal is sensitive and reachable before treating it as a leak.

Evidence

password = [literal redacted]

Recommendation: Review and address this secrets finding.

13. Published Demo Credentials

Risk	medium
Severity / priority	medium / medium
State	New
Module / category	static / credential_management
Finding ID	decb3276-4e95-4284-88b8-c2402ba33a74
Location	README.md:17
Created	2026-11-01T18:32:36.377Z

The README publishes email/password pairs that are presented as demo accounts for the runnable shop. Anyone with access to the repository can use these shared credentials to authenticate as those users, potentially exposing account-specific orders if the same accounts or data are available outside a disposable local environment. The supplied source does not establish deployment scope or whether these credentials are disabled in shared instances.

Evidence

Demo accounts (fictional, for this sample only):

Email	Password
`demo@example.com`	`demo-password-123`
`alex@example.com`	`alex-password-456`

Recommendation: Do not distribute working account passwords in repository documentation. Provision disposable local-only fixtures during setup, ensure they cannot authenticate in shared or production environments, and document placeholder credentials instead.

14. Unsupported Discount Variants Accepted

Risk	medium
Severity / priority	medium / medium
State	New
Module / category	static / business_rule_validation
Finding ID	6b096816-f891-481f-a1f2-c60ffbd893df
Location	src/orders.js:25
Created	2026-11-01T18:32:36.317Z

Discount codes are trimmed and uppercased before lookup, so values other than the specified exact `WELCOME10` string, such as `welcome10` or strings padded with whitespace, receive the discount instead of the mandated exact invalid-code response.

Evidence

```
const rate = DISCOUNT_CODES[String(discountCode).trim().toUpperCase()];
  if (!rate) return { error: 'This discount code is not valid.' };
```

Recommendation: Compare the submitted value directly with the exact supported code `WELCOME10`; reject every other supplied value with exactly `This discount code is not valid.` Add tests for lowercase, mixed-case, whitespace-padded, and unrelated codes.

15. Credential Error Enables Email Enumeration

Risk	medium
Severity / priority	medium / medium
State	New
Module / category	static / authentication
Finding ID	4b295795-8178-4ff4-88a7-2bca9d2f93f4
Location	server.js:45
Created	2026-11-01T18:32:36.236Z

Login returns a distinct message when no account matches the email, while a wrong password returns the required generic message. Because the UI displays the server message, an observer can distinguish registered from unregistered email addresses.

Evidence

```
if (!user) return sendJson(res, 401, { error: 'No account uses this email.' });
```

Recommendation: Return exactly `Email or password is incorrect.` with the same status and response shape for both unknown-email and wrong-password failures. Add API and UI tests asserting byte-for-byte identical messages for both cases.

16. Unvalidated Email Input Can Throw

Risk	medium
Severity / priority	medium / medium
State	New
Module / category	static / Input validation
Finding ID	abd301b2-5da4-481f-a4b0-c5f6b6bacf0c
Location	src/users.js:7
Created	2026-11-01T18:32:35.957Z

findUserByEmail calls trim() and toLowerCase() directly on the supplied email. A null, missing, or non-string value throws before the lookup completes, so malformed sign-in input can escape the intended credential-result path unless an unshown caller validates or catches it.

Evidence

```
function findUserByEmail(email) {
  return users.find(user => user.email === email.trim().toLowerCase()) || null;
```

Recommendation: Validate that email is a string before normalization and make malformed credentials follow the same controlled invalid-credential response path as other failed sign-ins.

17. User Enumeration Through Distinct Login Error

Risk	medium
Severity / priority	medium / medium
State	New
Module / category	static / authentication privacy
Finding ID	3921f473-7df8-42b1-9853-087c9a9a73d6
Location	server.js:45
Created	2026-11-01T18:32:35.719Z

The login endpoint returns a unique error when an email is absent, while a wrong password returns the required generic error. An unauthenticated requester can therefore determine which email addresses have accounts, contrary to the indistinguishable-error requirement.

Evidence

```
if (!user) return sendJson(res, 401, { error: 'No account uses this email.' });
```

Recommendation: Return exactly "Email or password is incorrect." for both unknown-email and incorrect-password cases.

18. Account Enumeration via Distinct Sign-in Error

Risk	medium
Severity / priority	medium / medium
State	New
Module / category	static / authentication_privacy
Finding ID	66832dd5-2e2d-40d7-b119-2fe33b16404c
Location	README.md:28
Created	2026-11-01T18:23:57.416Z

The documented login behavior reports that an email has no account, allowing unauthenticated users to distinguish unregistered email addresses. This violates the required identical invalid-credential response.

Evidence

| REQ-1 Sign-in | The login error says "No account uses this email", which tells strangers which emails are registered. |

Recommendation: Return exactly "Email or password is incorrect." for both unknown-email and incorrect-password failures, with the same externally observable status and response shape.

19. Unsupported Discount Variants Accepted

Risk	medium
Severity / priority	medium / medium
State	New
Module / category	static / discount_validation
Finding ID	d59709c1-1e80-4470-b8b0-6369ffdbaf9e
Location	src/orders.js:24
Created	2026-11-01T18:23:57.333Z

The checkout normalizes submitted discount codes to uppercase before lookup. As a result, values such as `welcome10` are accepted even though the confirmed rule states that any code other than the exact `WELCOME10` value must be rejected.

Evidence

```
if (discountCode) {
  const rate = DISCOUNT_CODES[String(discountCode).trim().toUpperCase()];
  if (!rate) return { error: 'This discount code is not valid.' };
```

Recommendation: Validate the submitted value against the exact supported code `WELCOME10` without case normalization. Return exactly `This discount code is not valid.` for every other nonempty value.

20. Account Enumeration Through Distinct Login Error

Risk	medium
Severity / priority	medium / medium
State	New
Module / category	static / authentication_privacy
Finding ID	a11306f4-ef8b-4471-9ef9-ff875f89faac
Location	server.js:45
Created	2026-11-01T18:23:57.162Z

The login endpoint returns a unique error when no account matches the submitted email, while a wrong password returns the required generic error. This lets callers distinguish registered from unregistered email addresses and violates the requirement that both cases display exactly the same message.

Evidence

```
if (!user) return sendJson(res, 401, { error: 'No account uses this email.' });
```

Recommendation: Return `{ error: 'Email or password is incorrect.' }` with the same status and materially equivalent behavior for both unknown-email and wrong-password failures.

21. [cq-console-log] Possible debug output call. Check its purpose against the project logging policy.

Risk	low
Severity / priority	low / low
State	New
Module / category	static / code_quality
Finding ID	50093a63-837f-480a-9c19-9e9d6d42990e
Location	server.js:107
Created	2026-11-01T18:32:35.527Z

Possible debug output call. Check its purpose against the project logging policy.

Evidence

```
console.log(`Sample Shop running at http://localhost:${PORT}`);
```

Recommendation: Review and address this code quality finding.

22. [cq-console-log] Possible debug output call. Check its purpose against the project logging policy.

Risk	low
Severity / priority	low / low
State	New
Module / category	static / code_quality
Finding ID	ff2e0d0c-76ee-40bf-a5b2-801927241620
Location	server.js:107
Created	2026-11-01T18:23:56.727Z

Possible debug output call. Check its purpose against the project logging policy.

Evidence

```
console.log(`Sample Shop running at http://localhost:${PORT}`);
```

Recommendation: Review and address this code quality finding.

23. [sec-innerhtml] HTML injection sink needs flow verification. This assignment alone does not establish an exploitable XSS vulnerability.

Risk	Not classified
Severity / priority	info / low
State	New
Module / category	static / security
Finding ID	89e1ff6a-c4d7-4cc7-8368-8cc270885a46
Location	public/app.js:19
Created	2026-11-01T18:32:35.356Z

HTML injection sink needs flow verification. This assignment alone does not establish an exploitable XSS vulnerability.

Evidence

```
item.innerHTML = `<span class="name"></span><span class="price">${money(product.price)}</span>`;
```

Recommendation: Trace the assigned value to its caller or input source. Verify the trust boundary, escaping and sanitization before promoting this observation to a vulnerability.

24. [sec-innerhtml] HTML injection sink needs flow verification. This assignment alone does not establish an exploitable XSS vulnerability.

Risk	Not classified
Severity / priority	info / low
State	New
Module / category	static / security
Finding ID	6e0d545f-2eb3-4295-85fa-87edabf8faf1
Location	public/app.js:19
Created	2026-11-01T18:23:56.590Z

HTML injection sink needs flow verification. This assignment alone does not establish an exploitable XSS vulnerability.

Evidence

```
item.innerHTML = `<span class="name"></span><span class="price">${money(product.price)}</span>`;
```

Recommendation: Trace the assigned value to its caller or input source. Verify the trust boundary, escaping and sanitization before promoting this observation to a vulnerability.

Latest Static Review attempt

Review	Pre-release review (Retry)
Review ID	f234a9ef-b148-43d3-90ef-ec1eaf039f06
Lifecycle state	Need approval
Execution status	Need approval
Decision	No decision recorded
Execution time	2026-11-01T18:28:58.169Z (creation-time fallback)
Created	2026-11-01T18:28:58.169Z
Last activity	2026-11-01T18:32:37.135Z
Decision time	Unavailable

Summary: Static review completed with 12 findings (3 deterministic, 9 model).

Reported findings

1. Order Ownership Check Missing

Risk	critical
Severity / priority	critical / high
State	New
Module / category	static / authorization
Finding ID	bf61ded6-9798-42b1-8d8d-4123a28fb176
Location	server.js:76
Created	2026-11-01T18:32:36.106Z

The authenticated order-view route looks up an order by ID and returns it without verifying that order.userId matches the signed-in userId. Any signed-in user who knows another order ID can receive that order's details, including the seeded shipping address, contrary to the order-privacy requirements.

Evidence

```
const order = findOrder(orderMatch[1]);
  if (!order) return sendJson(res, 404, { error: 'Order not found.' });
  return sendJson(res, 200, { order });
```

Recommendation: Before returning an order, require both that it exists and that order.userId equals the authenticated userId. Return exactly `{ error: 'Order not found.' }` with 404 for either a missing or foreign order, and add cross-user access tests verifying that no order fields are disclosed.

2. Server-Side Quantity Validation Missing

Risk	high
Severity / priority	high / medium
State	New
Module / category	static / input_validation
Finding ID	9682a57f-90f2-419f-b899-4754aa559a00
Location	server.js:65
Created	2026-11-01T18:32:36.170Z

Checkout converts each client-supplied quantity with Number() and stores it without checking that it is an integer from 1 through 10. Negative, fractional, excessive, or non-finite quantities can therefore reach order total calculation and corrupt the resulting order total.

Evidence

```
lines.push({ productId: product.id, name: product.name, price: product.price, quantity: Number(item.quantity) });
```

Recommendation: Parse each quantity once and reject it unless `Number.isInteger(quantity)` and `quantity >= 1 && quantity <= 10`. Perform this validation before creating any order and add server tests for zero, negative, fractional, over-10, NaN, and infinite values.

3. Missing Order Ownership Check

Risk	critical
Severity / priority	high / high
State	New
Module / category	static / authorization
Finding ID	47033209-ccaf-412f-8075-ef9ed93b329c
Location	server.js:76
Created	2026-11-01T18:32:35.868Z

The order-view route verifies only that some user is signed in. It retrieves an order solely by request-controlled ID and returns it without comparing order.userId to the authenticated user, exposing another user's line items, total, and potentially shipping details instead of returning exactly "Order not found.".

Evidence

```
const order = findOrder(orderMatch[1]);
  if (!order) return sendJson(res, 404, { error: 'Order not found.' });
  return sendJson(res, 200, { order });
```

Recommendation: After lookup, treat the order as not found unless order.userId exactly matches the authenticated userId, and return only the required 404 error without order details.

4. Missing Server-Side Quantity Validation

Risk	high
Severity / priority	high / medium
State	New
Module / category	static / checkout validation
Finding ID	821bad2d-85a5-407b-ab73-f6603cf454dc
Location	server.js:65
Created	2026-11-01T18:32:35.796Z

Checkout converts the submitted quantity to a Number but does not reject fractional, non-finite, zero, negative, or greater-than-ten values before creating the order. Crafted requests can therefore place orders with quantities outside the required whole-number range of 1 through 10 and can corrupt calculated totals.

Evidence

lines.push({ productId: product.id, name: product.name, price: product.price, quantity: Number(item.quantity) });

Recommendation: Before adding each line, require Number.isInteger(quantity) and quantity >= 1 && quantity <= 10; otherwise return a deterministic 400 response without creating an order.

5. [secrets-api-key] Possible hardcoded credential in "password". Verify whether this literal is sensitive and reachable before treating it as a leak.

Risk	critical
Severity / priority	high / high
State	New
Module / category	static / secrets
Finding ID	ef77f5a6-90ea-4af0-b090-86c5e2ea1707
Location	src/users.js:3
Created	2026-11-01T18:32:35.620Z

Possible hardcoded credential in "password". Verify whether this literal is sensitive and reachable before treating it as a leak.

Evidence

password = [literal redacted]

Recommendation: Review and address this secrets finding.

6. Published Demo Credentials

Risk	medium
Severity / priority	medium / medium
State	New
Module / category	static / credential_management
Finding ID	decb3276-4e95-4284-88b8-c2402ba33a74
Location	README.md:17
Created	2026-11-01T18:32:36.377Z

The README publishes email/password pairs that are presented as demo accounts for the runnable shop. Anyone with access to the repository can use these shared credentials to authenticate as those users, potentially exposing account-specific orders if the same accounts or data are available outside a disposable local environment. The supplied source does not establish deployment scope or whether these credentials are disabled in shared instances.

Evidence

Demo accounts (fictional, for this sample only):

Email	Password
`demo@example.com`	`demo-password-123`
`alex@example.com`	`alex-password-456`

Recommendation: Do not distribute working account passwords in repository documentation. Provision disposable local-only fixtures during setup, ensure they cannot authenticate in shared or production environments, and document placeholder credentials instead.

7. Unsupported Discount Variants Accepted

Risk	medium
Severity / priority	medium / medium
State	New
Module / category	static / business_rule_validation

Finding ID	6b096816-f891-481f-a1f2-c60ffbd893df
Location	src/orders.js:25
Created	2026-11-01T18:32:36.317Z

Discount codes are trimmed and uppercased before lookup, so values other than the specified exact `WELCOME10` string, such as `welcome10` or strings padded with whitespace, receive the discount instead of the mandated exact invalid-code response.

Evidence

```
const rate = DISCOUNT_CODES[String(discountCode).trim().toUpperCase()];
  if (!rate) return { error: 'This discount code is not valid.' };
```

Recommendation: Compare the submitted value directly with the exact supported code `WELCOME10`; reject every other supplied value with exactly `This discount code is not valid.` Add tests for lowercase, mixed-case, whitespace-padded, and unrelated codes.

8. Credential Error Enables Email Enumeration

Risk	medium
Severity / priority	medium / medium
State	New
Module / category	static / authentication
Finding ID	4b295795-8178-4ff4-88a7-2bca9d2f93f4
Location	server.js:45
Created	2026-11-01T18:32:36.236Z

Login returns a distinct message when no account matches the email, while a wrong password returns the required generic message. Because the UI displays the server message, an observer can distinguish registered from unregistered email addresses.

Evidence

```
if (!user) return sendJson(res, 401, { error: 'No account uses this email.' });
```

Recommendation: Return exactly `Email or password is incorrect.` with the same status and response shape for both unknown-email and wrong-password failures. Add API and UI tests asserting byte-for-byte identical messages for both cases.

9. Unvalidated Email Input Can Throw

Risk	medium
Severity / priority	medium / medium
State	New
Module / category	static / Input validation
Finding ID	abd301b2-5da4-481f-a4b0-c5f6b6bacf0c
Location	src/users.js:7
Created	2026-11-01T18:32:35.957Z

findUserByEmail calls trim() and toLowerCase() directly on the supplied email. A null, missing, or non-string value throws before the lookup completes, so malformed sign-in input can escape the intended credential-result path unless an unshown caller validates or catches it.

Evidence

```
function findUserByEmail(email) {
  return users.find(user => user.email === email.trim().toLowerCase()) || null;
```

Recommendation: Validate that email is a string before normalization and make malformed credentials follow the same controlled invalid-credential response path as other failed sign-ins.

10. User Enumeration Through Distinct Login Error

Risk	medium
Severity / priority	medium / medium

State	New
Module / category	static / authentication privacy
Finding ID	3921f473-7df8-42b1-9853-087c9a9a73d6
Location	server.js:45
Created	2026-11-01T18:32:35.719Z

The login endpoint returns a unique error when an email is absent, while a wrong password returns the required generic error. An unauthenticated requester can therefore determine which email addresses have accounts, contrary to the indistinguishable-error requirement.

Evidence

```
if (!user) return sendJson(res, 401, { error: 'No account uses this email.' });
```

Recommendation: Return exactly "Email or password is incorrect." for both unknown-email and incorrect-password cases.

11. [cq-console-log] Possible debug output call. Check its purpose against the project logging policy.

Risk	low
Severity / priority	low / low
State	New
Module / category	static / code_quality
Finding ID	50093a63-837f-480a-9c19-9e9d6d42990e
Location	server.js:107
Created	2026-11-01T18:32:35.527Z

Possible debug output call. Check its purpose against the project logging policy.

Evidence

```
console.log(`Sample Shop running at http://localhost:${PORT}`);
```

Recommendation: Review and address this code quality finding.

12. [sec-innerhtml] HTML injection sink needs flow verification. This assignment alone does not establish an exploitable XSS vulnerability.

Risk	Not classified
Severity / priority	info / low
State	New
Module / category	static / security
Finding ID	89e1ff6a-c4d7-4cc7-8368-8cc270885a46
Location	public/app.js:19
Created	2026-11-01T18:32:35.356Z

HTML injection sink needs flow verification. This assignment alone does not establish an exploitable XSS vulnerability.

Evidence

```
item.innerHTML = `<span class="name"></span><span class="price">${money(product.price)}</span>`;
```

Recommendation: Trace the assigned value to its caller or input source. Verify the trust boundary, escaping and sanitization before promoting this observation to a vulnerability.

Carryover findings

No carryover findings are persisted for this Review.

Recurring and other cross-Review classifications are unavailable for this record and are not inferred.

Source manifest

Status	available

Captured	2026-11-01T18:29:00.134Z
Artifacts reviewed	Unavailable

No source-manifest items are available.

Traceability

Snapshot status	available
Complete / incomplete / missing	0 / 0 / 16
Attention	16

- Validate item quantity on the server - missing; evidence: verified; mappings: 0; source artifacts: None
- Format prices in US dollars - missing; evidence: verified; mappings: 0; source artifacts: None
- REQ-2 Product catalogue - missing; evidence: verified; mappings: 0; source artifacts: None
- Calculate the order total - missing; evidence: verified; mappings: 0; source artifacts: None
- REQ-4 Order privacy - missing; evidence: verified; mappings: 0; source artifacts: None
- Exclude passwords from API responses - missing; evidence: verified; mappings: 0; source artifacts: None
- Restrict users to their own orders - missing; evidence: verified; mappings: 0; source artifacts: None
- Allow public catalogue access - missing; evidence: verified; mappings: 0; source artifacts: None
- Apply WELCOME10 discount - missing; evidence: verified; mappings: 0; source artifacts: None
- Use indistinguishable invalid-credential errors - missing; evidence: verified; mappings: 0; source artifacts: None
- REQ-1 Sign-in - missing; evidence: verified; mappings: 0; source artifacts: None
- Reject unsupported discount codes - missing; evidence: verified; mappings: 0; source artifacts: None
- Sign in with email and password - missing; evidence: verified; mappings: 0; source artifacts: None
- Conceal other users' orders - missing; evidence: verified; mappings: 0; source artifacts: None
- REQ-3 Checkout - missing; evidence: verified; mappings: 0; source artifacts: None
- Require sign-in to place orders - missing; evidence: verified; mappings: 0; source artifacts: None

Review model usage

Calls	9
Input / output tokens	243,777 / 25,144
Cache reads / creation	55,808 / 0

- codex / gpt-5.6-sol: 9 calls, 243,777 input, 25,144 output tokens

Latest existing Dynamic Test

Test	Sign-in page shows the login form
Test ID	d50762f9-6b3a-4f3e-a282-0fc4249cc1c0
Status	Completed
Target	http://localhost:4100
Goal	Open the shop and confirm that the sign-in form and the product list are shown. Do not sign in.
Created / updated	2026-11-01T18:37:41.557Z / 2026-11-01T18:37:53.144Z

Summary: Revision 1: 1 of 1 checks passed; verdict passed.

Evidence references

No evidence references are recorded.

Action trace

- 1. check - Expected visible text - passed: Configured criterion matched the observed page.

Raw console output and model request/response payloads are not included.

Project sources

Source artifacts recorded 10

- requirements.md - requirement / repository - docs/requirements.md - SHA-256 366c458be3ce04d4baf5bb02f92c827b53b84743eff87dd78a48ff33e8d554f6
- package.json - other / repository - package.json - SHA-256 bfc7eaaee4b7a1aa3394f61e63f06f55c8959195efd7fed3b9aed8b20c458079
- app.js - source_code / repository - public/app.js - SHA-256 06003bad2524f8583d03953794c30ed2db93153413506b5639d67a3ddca3c7af
- index.html - source_code / repository - public/index.html - SHA-256 3134fec8e57b1981a245182ea6f851e0fc571c168df3be2036412e013eafc81b
- styles.css - source_code / repository - public/styles.css - SHA-256 b4c5c0c863b4342fc6c61b73f992399bc9512833b7274757484c3992232cd718
- README.md - requirement / repository - README.md - SHA-256 ca18d54487b3bb176aa4f9ae96f53ca53ea642f74ef1a67f541f4320eb990e43
- server.js - source_code / repository - server.js - SHA-256 ee6726267cf1eb5feda4703b799f66e1adc70546cebfd01ac202f99ba7b4ad93
- orders.js - source_code / repository - src/orders.js - SHA-256 234e2efc64b73257256433756c7b52dc47b22c3804101e4cfbacf0277c3aae5c
- products.js - source_code / repository - src/products.js - SHA-256 21c1bf1ac814c290b74cec14141949aabee9073bcc8f9cfa8b17f10678da2bc5
- users.js - source_code / repository - src/users.js - SHA-256 f940a704ca83c309bd4ca096bdf542f1810493d1d72b0332042c747629e53fba

Requirements

REQ-3 Checkout

Requirement ID d9584da4-7ff1-4dc5-b6f4-36252c6d95d1

Category / priority requirement / medium

Only signed-in users may place orders; checkout must validate quantities server-side, calculate totals from item prices and quantities, and support only the specified WELCOME10 discount.

No source mappings are recorded.

Exclude passwords from API responses

Requirement ID 534b85ee-17a8-4685-933c-57448b67057a

Category / priority acceptance_criterion / medium

The API never returns passwords.

No source mappings are recorded.

Use indistinguishable invalid-credential errors

Requirement ID 7ed59005-12f1-4aae-99c9-538ad019f19a

Category / priority acceptance_criterion / medium

A wrong email and a wrong password both display exactly "Email or password is incorrect."

No source mappings are recorded.

Conceal other users' orders

Requirement ID	cf016c79-2b28-4913-bf84-be16578a03f8
Category / priority	acceptance_criterion / medium

When a user attempts to open another user's order, the response is exactly "Order not found." and contains none of that order's details.

No source mappings are recorded.

REQ-4 Order privacy

Requirement ID	43ab24ab-f3ef-420b-bf7a-464abc49afde
Category / priority	requirement / medium

Signed-in users must only be able to view their own orders, and attempts to open another user's order must reveal no order details.

No source mappings are recorded.

Apply WELCOME10 discount

Requirement ID	7dbe8fc7-024e-4708-82a9-4614f47c57b5
Category / priority	acceptance_criterion / medium

When the discount code `WELCOME10` is supplied, the order total is reduced by 10%.

No source mappings are recorded.

Require sign-in to place orders

Requirement ID	efaa3d2c-a0ee-47e2-8aed-0f3ca0b5b2b9
Category / priority	acceptance_criterion / medium

An order can be placed only by a signed-in user.

No source mappings are recorded.

Restrict users to their own orders

Requirement ID	54d7f5bd-73a9-48f7-93dd-fa515f4f1e20
Category / priority	acceptance_criterion / medium

A signed-in user can view only orders belonging to that user.

No source mappings are recorded.

Calculate the order total

Requirement ID	202d1fac-073a-4240-a154-01977a418874
Category / priority	acceptance_criterion / medium

The order total equals the sum, across every item, of its price multiplied by its quantity.

No source mappings are recorded.

REQ-2 Product catalogue

Requirement ID	1d53d377-ddcc-4b90-9743-b53c7203aaf8
Category / priority	requirement / medium

The product catalogue must be publicly viewable without sign-in and show product names and prices formatted in US dollars with two decimal places.

No source mappings are recorded.

REQ-1 Sign-in

Requirement ID	a0d40b52-e1d2-42f2-9946-d2e7d01f7ac3
Category / priority	requirement / medium

The product must support user sign-in with an email address and password, use an identical error message for wrong emails and wrong passwords, and never return passwords through the API.

No source mappings are recorded.

Validate item quantity on the server

Requirement ID	0412c95d-e689-4389-a524-2da12ef908ee
Category / priority	acceptance_criterion / medium

For every order item, the server accepts only whole-number quantities from 1 through 10 and rejects all other quantities regardless of browser-side behavior.

No source mappings are recorded.

Format prices in US dollars

Requirement ID	0e628a4a-7f60-4b47-89c8-95953fa3e40e
Category / priority	acceptance_criterion / medium

Every displayed price uses US dollars and exactly two decimal places.

No source mappings are recorded.

Reject unsupported discount codes

Requirement ID	c737e87d-76ee-44ef-83a8-077de2c1af01
Category / priority	acceptance_criterion / medium

Any discount code other than `WELCOME10` is rejected with exactly "This discount code is not valid."

No source mappings are recorded.

Allow public catalogue access

Requirement ID	627beccb-7cc4-47f1-ab6d-4b3233aca9dd
Category / priority	acceptance_criterion / medium

A user who is not signed in can view the product list, including product names and prices.

No source mappings are recorded.

Sign in with email and password

Requirement ID	cb4f51e2-03dc-47e3-a502-48cd9800fcb8
Category / priority	acceptance_criterion / medium

A user can sign in using an email address and password.

No source mappings are recorded.

Coding standards

No coding-standard artifacts are recorded.

Model usage summary

Calls	26
Input / output tokens	698,458 / 99,911
Cache reads / creation	128,512 / 0

- codex / gpt-5.6-sol: 26 calls, 698,458 input, 99,911 output tokens

Usage contains provider/model and token counts only. Credentials, prompts, model responses, and hidden reasoning are excluded.